

CSIT6910

Independent Project Report

R2T-Based Differentially Private Query Evaluation on TPC-H

Name: Xu Dongliu

Student ID: 21208710

Instructor: Ke Yi

Semester: 2025-26 Spring

May 26, 2026

Contents

1	Project Overview	2
1.1	Project Title	2
1.2	Background	2
1.3	Project Scope and Objectives	2
1.4	TPC-H Cases Used in the Project	3
2	Algorithm Design	4
2.1	Algorithm Overview	4
2.2	Algorithm Detail	5
2.3	Implementation Details	5
2.4	Q3-Style SQL Query	6
2.5	Q5-Style SQL Query	7
2.6	Implementation Structure	7
3	Experiment Design and Outcome	8
3.1	Experiment Overview	8
3.2	Input Design	9
3.3	Answer Check Design	10
3.4	Running Commands	10
3.5	Experiment Results	11
4	Conclusion	12
4.1	Conclusion	12
A	Sample of Minutes	13
A.1	Minutes of the 1st Project Meeting	13
A.2	Minutes of the 2nd Project Meeting	14
A.3	Minutes of the 3rd Project Meeting	15
A.4	Minutes of the 4th Project Meeting	16
A.5	Project code	17

Chapter 1

Project Overview

1.1 Project Title

R2T-Based Differentially Private Query Evaluation on TPC-H

1.2 Background

Differential privacy provides a formal way to release query answers while limiting how much one private entity can affect the output. In a single flat table, the sensitivity of a sum or count query is usually easy to bound. In a relational database, however, one entity can be connected to many secondary tuples through foreign keys. For example, in TPC-H one customer may have many orders, and each order may have many lineitems. If customer privacy is the target, removing one customer requires removing all referencing orders and lineitems, so a join query can change by a large amount.

The R2T paper, *R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys*, proposes a Race-to-the-Top mechanism for this setting. The core idea is to use a truncation threshold τ to limit the contribution of one private entity, but to choose this threshold in a differentially private way. Small thresholds produce low noise but high truncation bias. Large thresholds preserve more of the true answer but require larger privacy noise. R2T evaluates a geometric sequence of thresholds and returns the best noisy, penalized candidate.

This project applies the R2T idea to the TPC-H benchmark. The goal is not to build a general-purpose differentially private SQL engine. Instead, the project implements two concrete TPC-H revenue cases as Python scripts, records the intermediate R2T candidates, validates important invariants, and produces reproducible output files.

1.3 Project Scope and Objectives

The project has four main objectives:

1. Understand the R2T algorithm and how it applies to relational schemas with foreign-key relationships.
2. Select two TPC-H cases that are representative but still small enough to implement directly.

3. Implement the two cases as separate Python programs under a `code/` directory, without copying the reference notebook.
4. Save results and provide a validation path so the output can be checked and reproduced.

Both implemented cases use **Customer** as the private relation. The protected unit is one customer together with the orders and lineitems that reference that customer. The implemented queries are adapted from TPC-H Query 3 and Query 5. They keep the important join structure and selection predicates from the benchmark, but the final output is a scalar revenue aggregate suitable for a single R2T release.

1.4 TPC-H Cases Used in the Project

The first case is a Q3-style customer revenue query. It joins **Customer**, **Orders**, and **Lineitem**. The query keeps customers in the **BUILDING** market segment, orders before 1995-03-15, and lineitems shipped after 1995-03-15. The revenue term is $L_EXTENDEDPRICE * (1 - L_DISCOUNT)$.

The second case is a Q5-style ASIA revenue query. It joins **Customer**, **Orders**, **Lineitem**, **Supplier**, **Nation**, and **Region**. It keeps orders in 1994, restricts the region to ASIA, and requires the supplier and customer to come from the same nation.

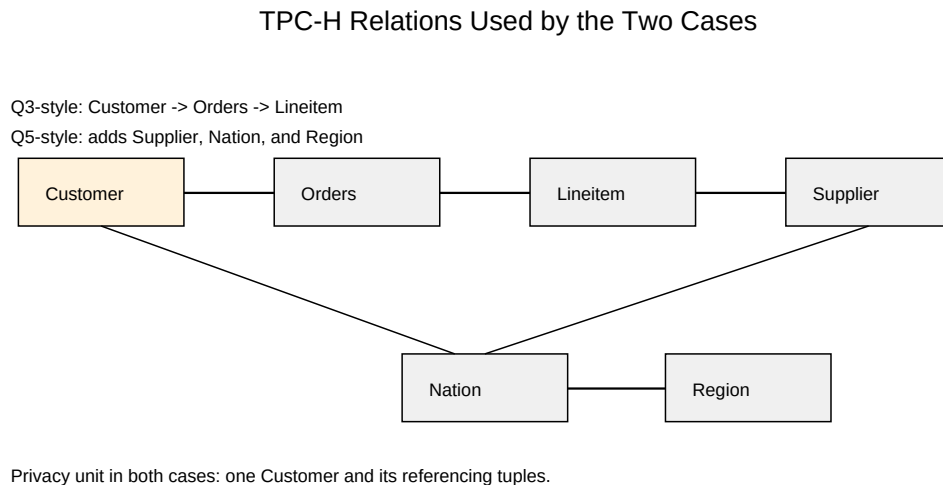


Figure 1.1: Foreign key relationship and TPC-H relations used in the project

The local TPC-H data files are stored under `BDT_IP.Differential-privacy/data`. The public GitHub repository does not include the raw `.tbl` files, because they are generated benchmark data and are significantly larger than the implementation.

Chapter 2

Algorithm Design

2.1 Algorithm Overview

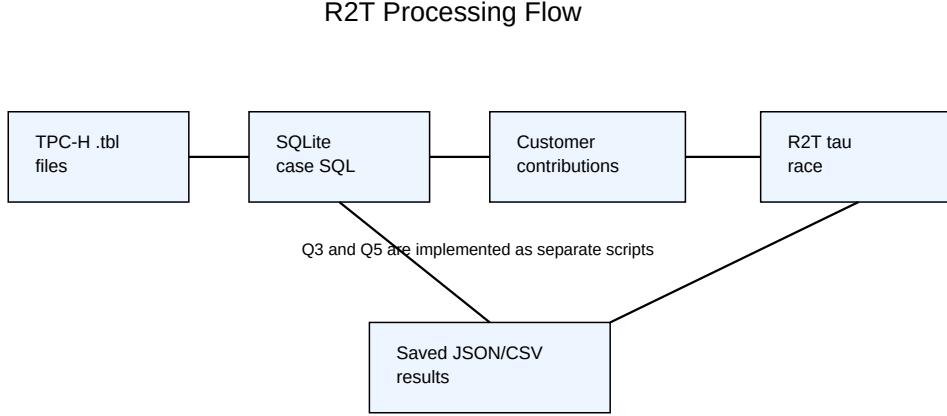


Figure 2.1: Algorithm flow

The implemented pipeline follows the structure shown in Figure 2.1. For each case, the program loads the required TPC-H `.tbl` files into an in-memory SQLite database. It then executes a case-specific SQL query that computes the revenue contribution for each customer. After this preprocessing step, the R2T mechanism is applied to the list of customer-level contributions.

Let x_i denote the contribution of customer i to the selected query. For a threshold τ , the truncated query answer is

$$Q(I, \tau) = \sum_i \min(x_i, \tau).$$

This form is valid for the two selected cases because they are self-join-free and the privacy unit is the customer. In the general R2T paper, a linear program is used for more complex

queries, especially self-joins or projections. For these two cases, the LP optimum reduces to the same per-customer truncation form, so an explicit LP solver is not needed.

2.2 Algorithm Detail

The project implements the following steps:

1. **Source Loading:** Read selected TPC-H `.tbl` files using Python’s standard `csv` module and insert them into an in-memory SQLite database.
2. **Case Query Execution:** Execute one fixed SQL query per case. The output is grouped by customer key and contains one revenue value for each contributing customer.
3. **Contribution Construction:** Convert the SQL output into a list of pairs (`customer_key`, `contribution`). The sum of all contributions is the non-private SQL answer.
4. **Threshold Enumeration:** Generate candidate thresholds $\tau = 2, 4, 8, \dots, 2^{\lfloor \log_2(GS_Q) \rfloor}$. The default value is $GS_Q = 10,000,000$.
5. **Truncated Answer:** For each threshold, compute $\sum_i \min(x_i, \tau)$. This value is monotone in τ and never exceeds the original SQL total.
6. **Noisy R2T Score:** Add Laplace noise and subtract the R2T penalty term. The score for threshold τ is

$$\tilde{Q}(I, \tau) = Q(I, \tau) + \text{Lap}\left(\frac{\log_2(GS_Q)\tau}{\epsilon}\right) - \log_2(GS_Q) \ln\left(\frac{\log_2(GS_Q)}{\beta}\right) \frac{\tau}{\epsilon}.$$

7. **Final Output:** Return

$$\max\left(0, \max_{\tau} \tilde{Q}(I, \tau)\right),$$

where $Q(I, 0) = 0$ because the revenue values are non-negative.

8. **Result Saving:** Save the summary output as JSON and all candidate thresholds as CSV files under `results/`.

2.3 Implementation Details

The implementation is divided into one shared utility module and two case-specific scripts. The shared module `code/common.py` is responsible for all steps that are common to both cases. The function `load_tables()` reads TPC-H `.tbl` files using the standard Python `csv` module. Since TPC-H rows are separated by the `|` character and end with a trailing separator, the loader inserts `row[:-1]` into SQLite so that the final empty field is removed. Each table is created with explicit TPC-H column names, which makes the SQL queries readable and avoids relying on column positions later in the program.

After the tables are loaded, each case script creates SQLite indexes on the main join keys. For the Q3-style case, the indexes are on `Orders(O_CUSTKEY)` and `Lineitem(L_ORDERKEY)`. For the Q5-style case, extra indexes are created on `Lineitem(L_SUPPKEY)` and `Supplier(S_NATIONKEY)`.

These indexes do not change the query result; they only make the full-data experiment faster.

The function `customer_contributions()` executes the SQL query and returns a list of (`customer_key`, `revenue`) pairs. This list is the bridge between SQL processing and R2T processing. The non-private SQL answer is simply the sum of these customer revenues. The private unit is a customer, so the contribution list is also the unit on which truncation is applied.

The function `r2t_from_contributions()` implements the actual R2T loop. It first computes $\log_2(\text{GS}_Q)$, then enumerates powers of two by calling `tau_candidates()`. For every threshold, it computes the truncated value with:

```
1 truncated = sum(min(value, tau) for _, value in contributions)
```

Then it samples Laplace noise using inverse transform sampling. A uniform random value is transformed into a Laplace random variable, so no external `numpy` dependency is required. The noisy R2T score is then computed as:

```
1 noise_scale = log_g * tau / epsilon
2 noisy_score = truncated + laplace(rng, noise_scale) -
  penalty_factor * tau
```

The best candidate is the row with the largest `noisy_score`. The final output applies the $Q(I, 0)$ lower bound:

```
1 r2t_output = max(best["noisy_score"], 0.0)
```

Finally, `save_result()` writes two output files. The JSON file stores the selected threshold, final R2T output, true SQL total, maximum customer contribution, and parameters. The CSV file stores all candidate thresholds, including the truncated answer, noise scale, and noisy score. This makes the final result reproducible and inspectable instead of only printing a terminal message.

2.4 Q3-Style SQL Query

```
1 SELECT
2     c.C_CUSTKEY,
3     SUM(l.L_EXTENDEDPRICE * (1.0 - l.L_DISCOUNT)) AS revenue
4 FROM Customer c
5 JOIN Orders o
6     ON c.C_CUSTKEY = o.O_CUSTKEY
7 JOIN Lineitem l
8     ON o.O_ORDERKEY = l.L_ORDERKEY
9 WHERE c.C_MKTSEGMENT = 'BUILDING'
10    AND o.O_ORDERDATE < '1995-03-15'
11    AND l.L_SHIPDATE > '1995-03-15'
12 GROUP BY c.C_CUSTKEY
13 HAVING revenue > 0;
```

This query is adapted from TPC-H Query 3. The original TPC-H Q3 reports the top revenue orders. In this project, the same filtering and join path are used, but the result is aggregated by customer because R2T protects customer-level contribution.

2.5 Q5-Style SQL Query

```
1 SELECT
2     c.C_CUSTKEY,
3     SUM(l.L_EXTENDEDPRICE * (1.0 - l.L_DISCOUNT)) AS revenue
4 FROM Customer c
5 JOIN Orders o
6     ON c.C_CUSTKEY = o.O_CUSTKEY
7 JOIN Lineitem l
8     ON o.O_ORDERKEY = l.L_ORDERKEY
9 JOIN Supplier s
10    ON l.L_SUPPKEY = s.S_SUPPKEY
11 JOIN Nation n
12    ON c.C_NATIONKEY = n.N_NATIONKEY
13    AND s.S_NATIONKEY = n.N_NATIONKEY
14 JOIN Region r
15    ON n.N_REGIONKEY = r.R_REGIONKEY
16 WHERE r.R_NAME = 'ASIA'
17    AND o.O_ORDERDATE >= '1994-01-01'
18    AND o.O_ORDERDATE < '1995-01-01'
19 GROUP BY c.C_CUSTKEY
20 HAVING revenue > 0;
```

This query is adapted from TPC-H Query 5. The original TPC-H Q5 groups by nation. This project keeps the ASIA-region and same-nation join logic but aggregates revenue by customer first, which is required to apply the customer-level R2T truncation.

2.6 Implementation Structure

The implementation is intentionally simple and direct. There is no query parser and no general SQL rewriting module. The files are:

- `code/common.py`: shared table loading, threshold generation, Laplace sampling, R2T scoring, printing, and result saving.
- `code/case_q3_customer_revenue.py`: the Q3-style case.
- `code/case_q5_asia_revenue.py`: the Q5-style case.
- `code/validate_cases.py`: lightweight validation checks.
- `code/generate_report_figures.py`: figure generation from saved CSV/JSON results.

The reference notebook was used only to understand the data loading style and the R2T loop. The submitted implementation does not copy the reference code and does not depend on `pandas`, `numpy`, or `pulp`.

Chapter 3

Experiment Design and Outcome

3.1 Experiment Overview

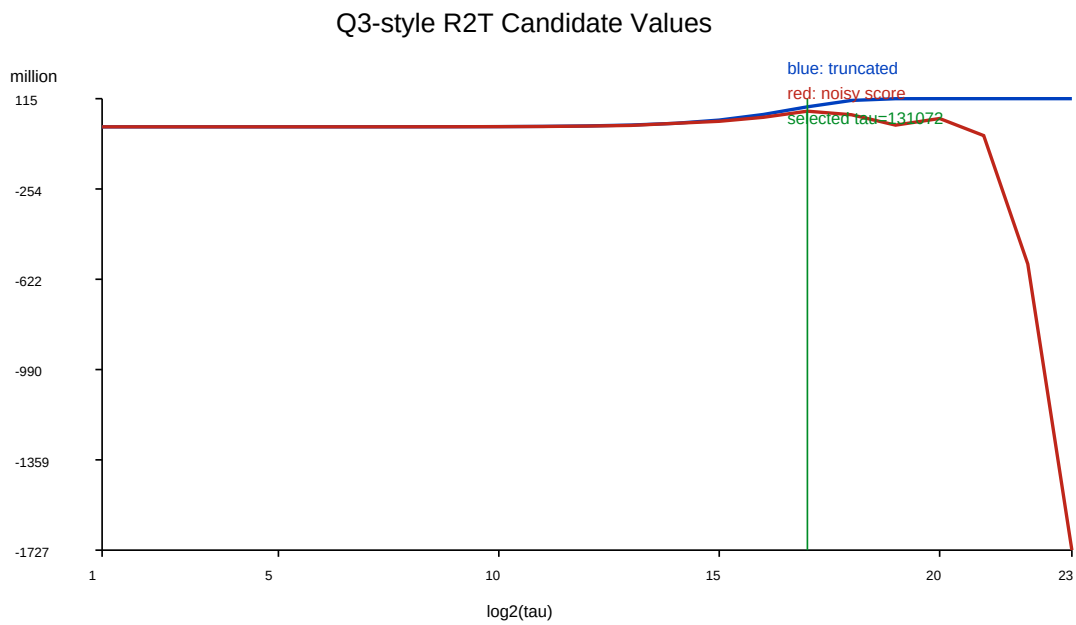


Figure 3.1: Experiment outcome for the Q3-style case

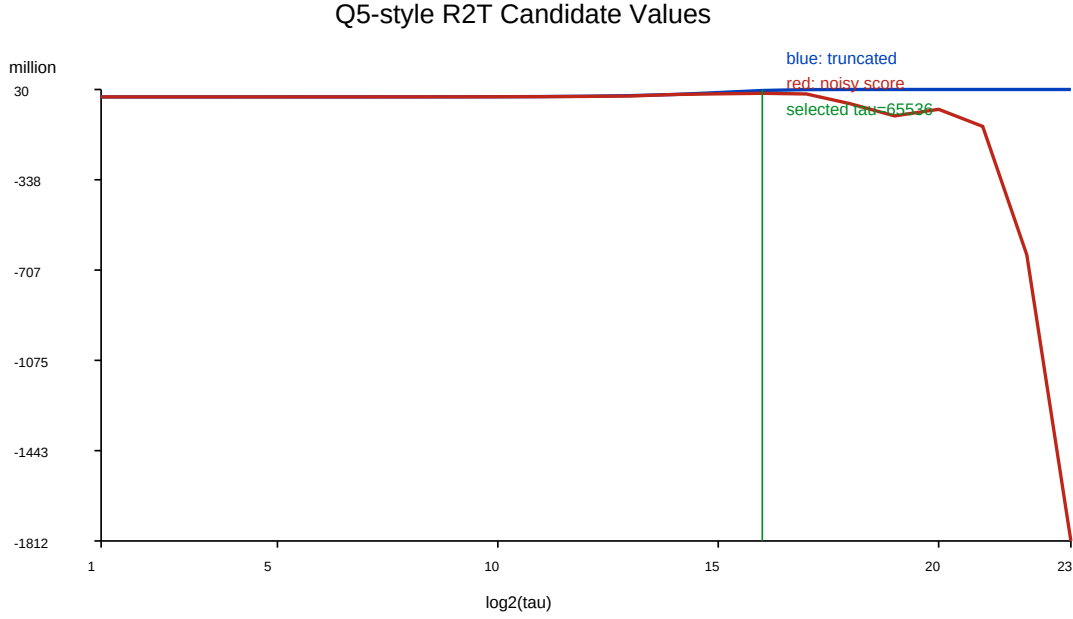


Figure 3.2: Experiment outcome for the Q5-style case

The experiment runs both implemented cases on the available TPC-H scale-1 data. For each case, the program outputs the true SQL total before privacy, the maximum customer contribution, the selected threshold τ , the truncated value at that threshold, and the final R2T noisy output. The candidate CSV files also record every threshold evaluated by the R2T race.

3.2 Input Design

1. Use the TPC-H .tbl files already available in `BDT_IP_Differential-privacy/data`. The tables used are `customer.tbl`, `orders.tbl`, `lineitem.tbl`, `supplier.tbl`, `nation.tbl`, and `region.tbl`.
2. Load only the tables required by each case. The Q3-style case loads `Customer`, `Orders`, and `Lineitem`. The Q5-style case also loads `Supplier`, `Nation`, and `Region`.
3. Store tables in SQLite memory tables with explicit column names based on the TPC-H schema. This allows the SQL query to be written clearly and keeps the project independent from external database services.
4. Create indexes on the main join keys, such as `O_CUSTKEY`, `L_ORDERKEY`, and `L_SUPPKEY`. These indexes reduce the runtime of the full data query.
5. Use fixed R2T parameters for reproducibility: $\epsilon = 0.8$, $\beta = 0.1$, $GS_Q = 10,000,000$, and random seed 1.

3.3 Answer Check Design

The project includes a validation script:

```
1 python3 code/validate_cases.py
```

The validation checks are not a replacement for a formal proof, but they verify important implementation properties:

1. Each case produces at least one contributing customer.
2. The true SQL total is positive.
3. The truncated value is monotone non-decreasing as τ increases.
4. The truncated value never exceeds the original SQL total.
5. The truncated value is bounded by $\tau \times$ the number of contributing customers.

These checks passed for both implemented cases. This gives confidence that the implementation follows the intended per-customer truncation logic and that the candidate files are internally consistent.

More specifically, the validation script reruns both cases on a smaller default sample and checks the following invariants inside `validate_result()`:

```
1 assert result["private_entities"] > 0
2 assert result["true_total"] > 0
3 assert result["best"]["tau"] > 0
4
5 previous = -1.0
6 for row in result["rows"]:
7     truncated = row["truncated"]
8     tau = row["tau"]
9     assert truncated >= previous
10    assert truncated <= result["true_total"] + 1e-6
11    assert truncated <= tau * result["private_entities"] + 1e-6
12    previous = truncated
```

The first three assertions check that the SQL case and R2T selection produce a meaningful result. The monotonicity check verifies the expected mathematical property of $\sum_i \min(x_i, \tau)$: increasing τ cannot decrease the truncated answer. The upper-bound check `truncated <= true_total` verifies that truncation never creates extra revenue. The final bound `truncated <= tau * private_entities` verifies the privacy-side constraint that each customer contributes at most τ after truncation. When the validation script was executed, both cases returned `ok`, confirming that these implementation-level checks passed.

3.4 Running Commands

The main commands used in the experiment are:

```

1 python3 code/case_q3_customer_revenue.py --seed 1
2 python3 code/case_q5_asia_revenue.py --seed 1
3 python3 code/validate_cases.py
4 python3 code/generate_report_figures.py

```

Each case writes two result files. The summary file records the selected threshold and final output, while the candidate file records every threshold evaluated by R2T.

3.5 Experiment Results

Metric	Q3-style	Q5-style
Contributing customers	904	655
True SQL total	114,904,912.53	30,276,617.68
Maximum customer contribution	661,967.22	210,018.26
Selected τ	131,072	65,536
Truncated value at selected τ	81,707,181.04	26,148,964.08
Noise scale at selected τ	3,809,852.89	1,904,926.45
R2T noisy output	63,926,157.02	14,653,924.70

Table 3.1: Full-data experimental results with seed 1

For the Q3-style case, the true SQL total is about 114.90 million. The largest individual customer contribution is about 0.66 million. R2T selects $\tau = 131,072$, which gives a truncated answer of about 81.71 million before noise and penalty. Larger thresholds preserve more of the true answer, but the penalty and noise scale also increase. The selected threshold is therefore smaller than the threshold that fully recovers the true answer.

For the Q5-style case, the true SQL total is about 30.28 million. The largest customer contribution is about 0.21 million. R2T selects $\tau = 65,536$, producing a truncated value of about 26.15 million. This case has fewer contributing customers and a smaller revenue scale than the Q3-style case, so the selected threshold and noise scale are also smaller.

Chapter 4

Conclusion

4.1 Conclusion

This project successfully applied the R2T truncation mechanism to two TPC-H revenue queries. The implementation protects customer-level contribution, computes per-customer revenue, evaluates R2T threshold candidates, adds calibrated Laplace noise and penalty terms, and saves all outputs for later inspection.

The main result is that R2T selects intermediate thresholds rather than the largest possible threshold. This behavior matches the intuition of the algorithm: increasing τ reduces truncation bias but increases noise and penalty. The final selected threshold balances these two effects.

The project is intentionally limited to two concrete cases. It does not try to support arbitrary SQL, self-joins, or projection-heavy SPJA queries. However, it demonstrates the central R2T workflow clearly on real TPC-H data and provides a reproducible code path, saved result files, generated figures, and a public GitHub repository.

Appendix A

Sample of Minutes

A.1 Minutes of the 1st Project Meeting

Meeting Minutes

Date: Saturday, February 7, 2026
Time: 4:00 pm
Place: Zoom meeting
Present: Xu Dongliu
Apology: None
Note-taker: Xu Dongliu

1. Approval of Minutes

This was the first project meeting, so there were no minutes to approve.

2. Discussion Items

Background Of Project

We discussed the R2T paper and how to use TPC-H data for a differential privacy project. The initial plan was to understand the foreign-key privacy model, inspect the reference notebook, and decide which TPC-H queries could be implemented within the project scope.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 4:45 pm. The next meeting was planned for March.

A.2 Minutes of the 2nd Project Meeting

Meeting Minutes

Date: Tuesday, March 17, 2026
Time: 3:00 pm
Place: Zoom meeting
Present: Xu Dongliu
Apology: None
Note-taker: Xu Dongliu

1. Approval of Minutes

The minutes of the first meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the project progress. The TPC-H schema and R2T truncation mechanism had been reviewed. The project direction was adjusted to implement two concrete TPC-H cases rather than a general query framework. The selected privacy relation was Customer.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 3:45 pm. The next meeting was planned for April.

A.3 Minutes of the 3rd Project Meeting

Meeting Minutes

Date: Tuesday, April 21, 2026
Time: 4:30 pm
Place: Zoom meeting
Present: Xu Dongliu
Apology: None
Note-taker: Xu Dongliu

1. Approval of Minutes

The minutes of the second meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the implementation progress. The Q3-style and Q5-style SQL queries had been written, and the code could load TPC-H tables into SQLite. The next step was to finish the R2T threshold loop, save outputs, and add validation checks.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 5:00 pm. The next meeting was planned for May.

A.4 Minutes of the 4th Project Meeting

Meeting Minutes

Date: Tuesday, May 19, 2026
Time: 3:00 pm
Place: Zoom meeting
Present: Xu Dongliu
Apology: None
Note-taker: Xu Dongliu

1. Approval of Minutes

The minutes of the third meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the experiment outcome and final report. The two R2T cases had been executed on the full TPC-H data. The validation script passed, figures were generated, and the public GitHub repository was prepared for submission.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 5:00 pm. This was the last meeting.

A.5 Project code

<https://github.com/xd12003/r2t-tpch-dp>